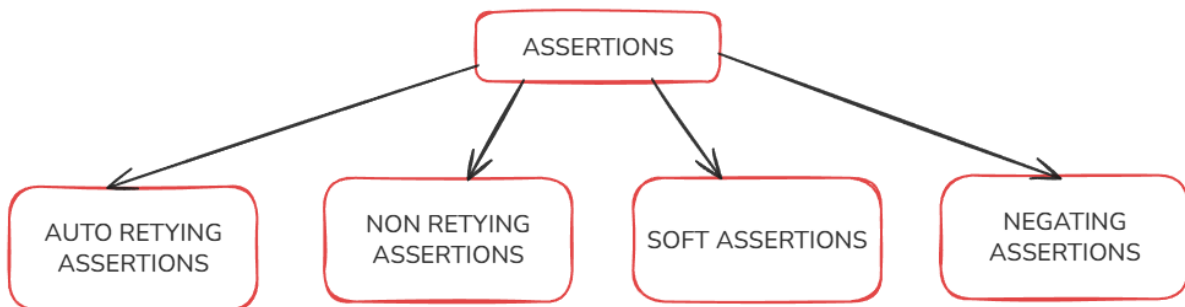


PLAYWRIGHT WITH TYPESCRIPT (PART B)

ASSERTIONS:



Assertions in Playwright are used to verify that the application is behaving as expected. When Playwright performs actions like opening a website, clicking a button, or entering text, assertions help confirm that the result of those actions is correct. Without assertions, Playwright would only perform the steps but would not check whether the outcome is right or wrong.

In Playwright, assertions are written using the `expect()` function. This function compares the actual result with the expected result.

SYNTAX:

```
await expect(locator_or_page).assertionMethod(expectedValue);
```

Here, `expect()` is used to perform the verification, and the assertion method defines what needs to be checked.

Auto-Retrying Assertions:

Auto-retrying assertions are one of the most powerful features of Playwright. They automatically wait and retry until the expected condition becomes true or the timeout is reached. This means you do not need to manually add waits like `setTimeout` or `waitForTimeout` in most cases.

In real applications, elements do not appear instantly. For example, after clicking a login button, the dashboard may take a few seconds to load. If Playwright checks immediately without waiting, the test may fail even though the application is working correctly. Auto-retrying assertions solve this problem by continuously checking the condition until it becomes true.

This automatic retry happens because Playwright understands that modern web applications are dynamic and elements may load after some delay.

By default, Playwright waits up to 5 seconds for an assertion to pass. If needed, you can increase the timeout.

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Auto-retrying assertions example with login form', async ({ page }) => {

  // Step 1: Open Playwright practice website
  await page.goto('https://practicetestautomation.com/practice-test-login/');

  // Step 2: Verify URL (auto-retrying assertion)
  await expect(page).toHaveURL('https://practicetestautomation.com/practice-test-login/');

  // Step 3: Verify username field is visible
  const username = page.locator('#username');
  await expect(username).toBeVisible();

  // Step 4: Verify password field is visible
  const password = page.locator('#password');
  await expect(password).toBeVisible();

  // Step 5: Enter username
  await username.fill('student');

  // Step 6: Verify entered username value (auto-retrying)
  await expect(username).toHaveValue('student');

  // Step 7: Enter password
  await password.fill('Password123');

  // Step 8: Verify entered password value
  await expect(password).toHaveValue('Password123');

  // Step 9: Click login button
  const loginButton = page.locator('#submit');
  await expect(loginButton).toBeVisible();
  await loginButton.click();

  // Step 10: Verify successful login URL
  await expect(page).toHaveURL('https://practicetestautomation.com/logged-in-successfully/');

  // Step 11: Verify success message is visible
  const successMessage = page.locator('.post-title');
  await expect(successMessage).toBeVisible();

  // Step 12: Verify success message text
  await expect(successMessage).toHaveText('Logged In Successfully');

});
```

Non-Retrying Assertions

Non-retrying assertions are assertions that check the condition only once and do not wait or retry if the condition is not true. Playwright immediately evaluates the value and gives the result. If the condition is true, the test passes. If the condition is false, the test fails instantly.

These assertions are used when you want to verify values that are already available immediately, such as numbers, strings, boolean values, or variables. Non-retrying assertions are commonly used for validating data, calculations, API responses, or values stored in variables.

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Example of non-retrying assertions', async ({ page }) => {

  // Step 1: Open the website
  await page.goto('https://example.com');

  // Step 2: Store the page title in a variable
  const pageTitle = await page.title();

  // Step 3: Non-retrying assertion to verify title
  expect(pageTitle).toBe('Example Domain');

  // Step 4: Store the page URL in a variable
  const pageURL = page.url();

  // Step 5: Non-retrying assertion to verify URL
  expect(pageURL).toBe('https://example.com/');

  // Step 6: Create a variable with number value
  const total = 10 + 5;

  // Step 7: Non-retrying assertion to verify number
  expect(total).toBe(15);

  // Step 8: Create a boolean variable
  const isLoginSuccessful = true;

  // Step 9: Non-retrying assertion to verify boolean value
  expect(isLoginSuccessful).toBeTruthy();

});
```

Soft Assertions

Soft assertions are used when you want the test to continue executing even if an assertion fails. In normal assertions, if any assertion fails, Playwright immediately stops the test and marks it as failed. But in soft assertions, Playwright records the failure and continues executing the remaining steps of the test.

This is useful when you want to verify multiple things in a single test and see all the failures together instead of stopping at the first failure.

In Playwright, soft assertions are written using `expect.soft()` instead of `expect()`.

The difference is that `expect()` stops the test on failure, while `expect.soft()` does not stop the test.

SYNTAX:

```
await expect.soft(locator_or_value).assertionMethod(expectedValue);
```

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Example of soft assertions', async ({ page }) => {

  // Step 1: Open the website
  await page.goto('https://example.com');

  // Step 2: Soft assertion to verify page title
  await expect.soft(page).toHaveTitle('Wrong Title');

  // Step 3: Soft assertion to verify URL
  await expect.soft(page).toHaveURL('https://example.com/');

  // Step 4: Store value in variable
  const number = 10;

  // Step 5: Soft assertion to verify number
  expect.soft(number).toBe(20);

  // Step 6: This step will still execute even if above assertions fail
  console.log('Test execution continues even after soft assertion failure');

});
```

Negating Assertions

Negating assertions are used to verify that something is **not true** or **does not exist**. In simple words, negating assertions check the opposite condition. Instead of checking that something is present, visible, or equal, negating assertions check that something is not present, not visible, or not equal.

In Playwright, negating assertions are written using `.not` with the `expect()` function.

The keyword `.not` tells Playwright to verify the opposite condition.

For example, if `toBeVisible()` checks that an element is visible, then `not.toBeVisible()` checks that the element is not visible.

SYNTAX:

```
await expect(locator_or_value).not.assertionMethod(expectedValue);
```

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Example of negating assertions', async ({ page }) => {

  // Step 1: Open the website
  await page.goto('https://example.com');

  // Step 2: Verify page title is NOT incorrect
  await expect(page).not.toHaveTitle('Facebook');

  // Step 3: Store page URL in variable
  const pageURL = page.url();

  // Step 4: Non-retrying negating assertion
  expect(pageURL).not.toBe('https://google.com');

  // Step 5: Locate heading element
  const heading = page.locator('h1');

  // Step 6: Verify heading is NOT hidden
  await expect(heading).not.toBeHidden();

});
```

Custom Messages in Assertions (Playwright with Typescript)

Custom messages in assertions are used to display your own message when an assertion fails. These messages help you understand the reason for the failure more clearly. Instead of showing only the default Playwright error, you can provide a meaningful message that explains what you were trying to verify.

This is very useful in real projects where tests are large and contain many assertions. Custom messages make debugging easier because the report clearly shows what went wrong.

In Playwright, you can add a custom message as the second argument to the `expect()` function.

SYNTAX:

```
await expect(locator_or_value, 'Your custom message').assertionMethod(expectedValue);
```

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Example of custom assertion messages', async ({ page }) => {

  // Step 1: Open the website
  await page.goto('https://example.com');

  // Step 2: Assertion with custom message for title
  await expect(
    page,
    'Page title is incorrect. Expected title should be Example Domain'
  ).toHaveTitle('Example Domain');

  // Step 3: Locate heading element
  const heading = page.locator('h1');

  // Step 4: Assertion with custom message for visibility
  await expect(
    heading,
    'Heading is not visible on the page'
  ).toBeVisible();

  // Step 5: Store URL in variable
  const url = page.url();

  // Step 6: Non-retrying assertion with custom message
  expect(
    url,
    'URL does not match the expected value'
  ).toBe('https://example.com/');

});
```

Wait Statements

Wait statements in Playwright are used to pause the test execution until a certain condition is met. In real web applications, elements do not load instantly. Sometimes elements take time to appear, disappear, or update. Wait statements help Playwright handle these situations properly.

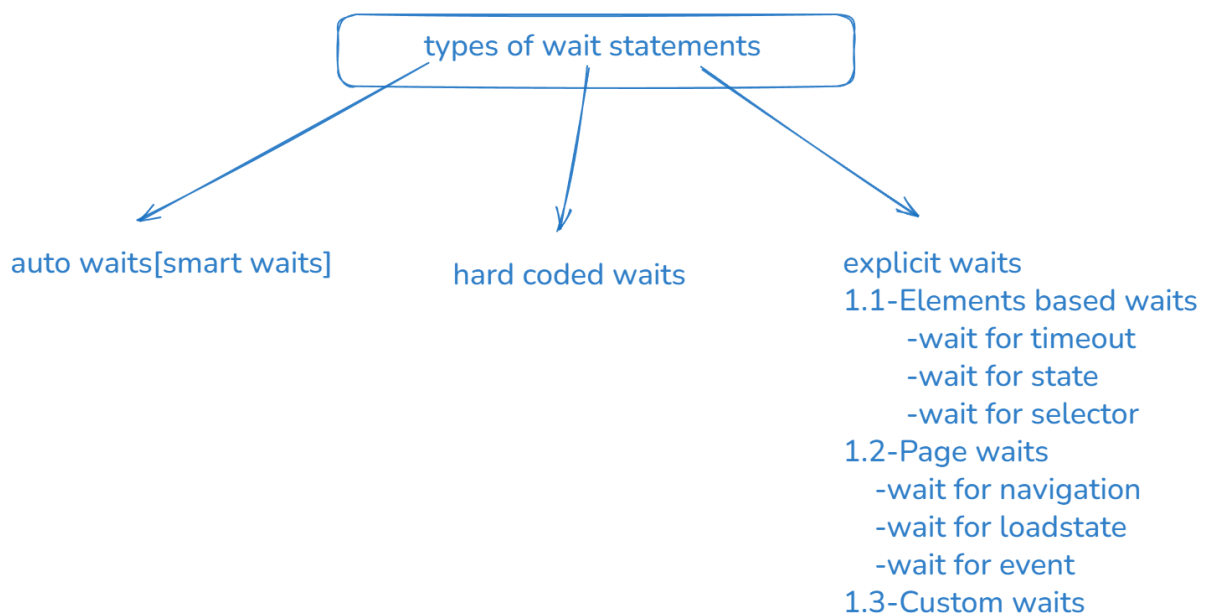
Playwright is designed with automatic waiting, so in most cases you do not need to add manual waits. However, in some situations, wait statements are useful when you need to wait for an element, URL, load state, or a fixed amount of time.

#WITHOUT WAIT STATEMENTS YOUR TEST SCRIPTS MIGHT??

- Try to click a button before it becomes clickable.
- Attempt to read text from an element that hasn't been loaded yet.
- Fail due to transitions , animations or slow network responses.

#WAIT STATEMENTS HELPS TO??

- Reduce flakiness in tests.
- Ensure consistency across different environments
- Improve reliability and reflect actual user behaviour



Auto Waits

Auto waits in Playwright mean that Playwright automatically waits for elements to be ready before performing any action on them. You do not need to manually add wait statements in most cases. Playwright intelligently waits for the element to become visible, enabled, stable, and ready for interaction.

This makes Playwright faster, more reliable, and easier to use compared to other automation tools where manual waits are often required.

For example, if you try to click a button, Playwright will automatically wait until the button is visible and clickable. If the button takes time to load, Playwright waits automatically instead of failing immediately.

Playwright automatically waits before performing actions such as:

- `click()`
- `fill()`
- `press()`
- `check()`
- `selectOption()`

In Playwright, you can change the default timeout in **four main ways**. The timeout defines how long Playwright will wait for an action or assertion before failing the test.

Playwright has different timeout levels such as test timeout, action timeout, navigation timeout, and assertion timeout. You can change these timeouts depending on your requirement.

1. Changing timeout in playwright.config.js (Global timeout)

This method changes the timeout for all tests globally. It is the most commonly used method in real projects.

EXAMPLE:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  timeout: 60000, // 60 seconds for each test

  use: {
    actionTimeout: 15000, // 15 seconds for actions like click, fill
    navigationTimeout: 20000 // 20 seconds for navigation
  }

});
```

2. Changing timeout inside a test using test.setTimeout()

This method changes the timeout for a specific test.

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Example with custom test timeout', async ({ page }) => {

  test.setTimeout(60000); // 60 seconds for this test only

  await page.goto('https://example.com');

  await expect(page).toHaveTitle('Example Domain');

});
```

3. Changing timeout for a specific action or assertion

You can change timeout for individual actions or assertions.

Example:

```
await page.click('#loginButton', { timeout: 10000 });
```

4. Changing timeout using page.setDefaultTimeout()

This method sets the timeout for all actions on a specific page.

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Example using setDefaultTimeout', async ({ page }) => {

  page.setDefaultTimeout(20000); // 20 seconds for all actions on this page

  await page.goto('https://example.com');

  await page.click('text=More information');

});
```

Hard Coded Waits

Hard coded waits are used to pause the test execution for a fixed amount of time. This means Playwright will stop execution and wait for the exact time you specify, regardless of whether the element is ready or not.

In Playwright, hard coded waits are implemented using the `waitForTimeout()` method. The time is given in milliseconds.

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Example of hard coded wait', async ({ page }) => {

  // Step 1: Open login page
  await page.goto('https://practicetestautomation.com/practice-test-login/');

  // Step 2: Wait for 3 seconds (hard coded wait)
  await page.waitForTimeout(3000);

  // Step 3: Enter username
  await page.fill('#username', 'student');

  // Step 4: Enter password
  await page.fill('#password', 'Password123');

  // Step 5: Wait for 2 seconds
  await page.waitForTimeout(2000);

  // Step 6: Click login button
  await page.click('#submit');

  // Step 7: Wait for 3 seconds
  await page.waitForTimeout(3000);

  // Step 8: Verify success message
  await expect(page.locator('.post-title')).toHaveText('Logged In Successfully');

});
```

DISADVANTAGES:

- Suppose the timeout is not enough for the elements to be so it will create flakiness

- Suppose your elements are loaded before the timeout then you have to wait till the hard coded timeout leading to increase processing time.
- Not flexible

Explicit Waits in Playwright (Typescript)

Explicit waits are used when you tell Playwright to wait for a specific condition to happen before continuing the execution. Instead of waiting for a fixed time like hard coded waits, explicit waits wait until a particular element appears, disappears, or reaches a certain state.

This makes explicit waits smarter and more efficient because Playwright waits only as long as necessary. If the condition happens quickly, Playwright continues immediately. If it takes longer, Playwright waits until the condition is satisfied or the timeout is reached.

Explicit waits are useful when elements load dynamically, after clicking a button, after navigation, or after some background process completes.

EXPLICIT WAITS SOLVES THIS BY:-

- Ensuring elements are present , visible or in a correct state
- Waiting for page navigation or network to complete
- Verifying text or data appears dynamically

Wait For Timeout:

Waits for a specific timeout with element state (like visible, attached)

Example :

```
await expect(locator).toBeVisible({ timeout: timeoutInMilliseconds });
```

waitForSelector()

The `waitForSelector()` method is used to make Playwright wait until a specific element appears on the webpage. This is an explicit wait, which means Playwright waits for a real condition instead of waiting for a fixed time.

Example:

```
await page.waitForSelector('selector', { timeout: 10000 });
```

Wait for Element State:

The `waitFor({ state })` method is used with locators to wait until an element reaches a specific state. It is an explicit wait that ensures the element is ready before Playwright performs any action on it.

This method is commonly used with locators and is the modern and recommended way to wait for element conditions in Playwright.

The most commonly used states are:

- "visible" → waits until the element becomes visible
- "hidden" → waits until the element becomes hidden
- "attached" → waits until the element is present in the DOM
- "detached" → waits until the element is removed from the DOM

Example:

```
await username.waitFor({ state: 'visible' });
```

Page Waits

Page waits in Playwright are used to wait for page-level events such as page loading, navigation, or network activity. These waits ensure that the page is fully loaded and ready before Playwright performs the next action.

When a page loads, different things happen such as HTML loading, resources loading, and network requests completing. Page wait methods help Playwright wait for these processes to finish.

Playwright provides page wait methods such as:

- `page.waitForNavigation()`
- `page.waitForLoadState()`
- `page.waitForEvent()`

These methods help ensure that the page is ready before interacting with elements

`waitForNavigation()`

The `waitForNavigation()` method is used to wait until the page navigates to a new page. Navigation happens when you click a link, click a button, submit a form, or perform any action that loads a new URL.

This method ensures that Playwright waits for the new page to load completely before performing the next action. Without this wait, Playwright may try to interact with the new page before it is fully loaded, which can cause test failures.

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Example of waitForNavigation', async ({ page }) => {

  // Step 1: Open login page
  await page.goto('https://practicetestautomation.com/practice-test-login/');

  // Step 2: Enter username
  await page.fill('#username', 'student');

  // Step 3: Enter password
  await page.fill('#password', 'Password123');

  // Step 4: Click login and wait for navigation
  await Promise.all([
    page.waitForNavigation(),
    page.click('#submit')
  ]);

  // Step 5: Verify new page URL
  await expect(page).toHaveURL('https://practicetestautomation.com/logged-in-successfully/');

});
```

waitForLoadState()

The `waitForLoadState()` method is used to wait until the page reaches a specific loading state. When a webpage loads, it goes through different stages such as loading HTML, loading resources, and completing network requests. This method ensures that Playwright waits until the page is ready before performing the next action.

This is a page-level wait, which means it waits for the entire page instead of a specific element.

The most commonly used load states are:

- "domcontentloaded" → waits until the HTML document is loaded
- "load" → waits until the entire page and its resources are loaded
- "networkidle" → waits until all network requests are finished

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Example of waitForLoadState', async ({ page }) => {

  // Step 1: Open the website
  await page.goto('https://example.com');

  // Step 2: Wait until HTML is loaded
  await page.waitForLoadState('domcontentloaded');

  // Step 3: Wait until full page is loaded
  await page.waitForLoadState('load');

  // Step 4: Wait until network requests are finished
  await page.waitForLoadState('networkidle');

  // Step 5: Verify page title
  await expect(page).toHaveTitle('Example Domain');

});
```

waitForEvent()

The `waitForEvent()` method is used to wait until a specific event happens on the page or browser. Events are actions such as opening a new tab, downloading a file, showing a popup, or closing a page. This method ensures that Playwright waits for the event before continuing execution.

This is useful when an action triggers something asynchronously, such as clicking a button that opens a new tab or starts a download.

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Example of waitForEvent for new tab', async ({ page }) => {

  // Step 1: Open example website
  await page.goto('https://example.com');

  // Step 2: Wait for new tab event and click link
  const newPage = await Promise.all([
    page.waitForEvent('popup'),
    page.click('a') // this link opens new tab
  ]);

  // Step 3: Get the new tab page object
  const popupPage = newPage[0];

  // Step 4: Wait for new tab to load
  await popupPage.waitForLoadState('load');

  // Step 5: Verify title of new tab
  const title = await popupPage.title();
  expect(title).toBe('Example Domain');

});
```

Custom Waits

Custom waits are used when you want Playwright to wait for a specific condition defined by you. This means instead of waiting for an element or page load, Playwright waits until your custom logic becomes true.

Custom waits are useful when there is no direct selector or built-in wait method available. For example, waiting for text to change, waiting for a value to update, or waiting for a Typescript condition to become true.

Playwright provides the `waitForFunction()` method to implement custom waits.

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Example of custom wait using waitForFunction', async ({ page }) => {

  // Step 1: Open example website
  await page.goto('https://example.com');

  // Step 2: Custom wait until heading text is available
  await page.waitForFunction(() => {
    const heading = document.querySelector('h1');
    return heading && heading.innerText === 'Example Domain';
  });

  // Step 3: Verify heading text
  const headingText = await page.locator('h1').textContent();
  expect(headingText).toBe('Example Domain');

});
```

Mouse Actions

Mouse actions in Playwright are used to simulate real user interaction with the application using the mouse. These actions include clicking, double clicking, right clicking, hovering, and drag and drop. Playwright provides two ways to perform mouse actions. The first way is using locator-based mouse actions such as `click()`, `dblclick()`, and `hover()`, which are used to interact directly with elements. The second way is using the **Mouse class**, which allows you to control the mouse cursor using screen coordinates.

Locator-based mouse actions are the most commonly used because they work directly with elements and Playwright automatically waits for the element to be visible, enabled, and ready before performing the action. For example, clicking a button or hovering over an element can be done easily using locator methods.

The Mouse class, accessed using `page.mouse`, is used when you need more precise control over the mouse cursor. It allows you to move the mouse to specific coordinates, click at exact positions, and perform drag and drop

operations. This is useful when working with canvas elements, graphical applications, or custom UI components where locators may not be available.

click() in Playwright

The `click()` method is used to perform a single mouse click on an element. This action simulates a real user clicking on a button, link, checkbox, or any clickable element. Playwright automatically waits for the element to be visible, enabled, and ready before performing the click.

This method is commonly used to submit forms, open links, or trigger actions in the application.

dblclick() in Playwright

The `dblclick()` method is used to perform a double click on an element. This simulates a user clicking the mouse twice quickly on the same element. This action is useful when applications require double click to edit, open, or select items.

hover() in Playwright

The `hover()` method is used to move the mouse pointer over an element. This simulates placing the mouse cursor on the element. This action is useful when menus, tooltips, or hidden options appear only when the mouse is over the element.

mouse.down() in Playwright

The `mouse.down()` method is used to press the mouse button at the current mouse position. This simulates pressing and holding the mouse button. It is mainly used in drag and drop operations.

mouse.up() in Playwright

The `mouse.up()` method is used to release the mouse button at the current mouse position. This simulates releasing the mouse button. It is used together with `mouse.down()` to complete drag and drop operations.

EXAMPLE:

```
// click()
await page.locator('#loginButton').click();

// dblclick()
await page.locator('#fileItem').dblclick();

// hover()
await page.locator('#menu').hover();

// mouse.down()
await page.mouse.down();

// mouse.up()
await page.mouse.up();
```

mouse.wheel()

The `mouse.wheel()` method is used to scroll the page using the mouse wheel. It simulates the action of scrolling up or down with the mouse. This method is useful when elements are not visible on the screen and you need to scroll to reach them.

-deltaX is horizontal scroll

- deltaY is vertical scroll

Positive value scrolls down, negative value scrolls up.

Example:

```
await page.mouse.wheel(0, 500);
```

scrollIntoViewIfNeeded()

The scrollIntoViewIfNeeded() method is used to scroll the page until the element becomes visible. Playwright automatically scrolls the element into view only if it is not already visible.

This method is useful when the element is present but outside the visible screen area.

Example:

```
await page.locator('#submitButton').scrollIntoViewIfNeeded();
```

dragTo()-

The dragTo() method is used to perform drag and drop from one element to another element. This simulates the real user action of clicking on an element, dragging it, and dropping it on another element.

This method is used with locators, where one locator is the source element and the other locator is the target element.

SYNTAX:

```
await sourceLocator.dragTo(targetLocator);
```

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Example of dragTo()', async ({ page }) => {

  // Open drag and drop website
  await page.goto('https://demo.playwright.dev/drag-and-drop');

  // Locate source element
  const source = page.locator('#item-1');

  // Locate target element
  const target = page.locator('#item-2');

  // Perform drag and drop
  await source.dragTo(target);

});
```

Keyboard Actions in Playwright (Typescript)

Keyboard actions in Playwright are used to simulate typing and pressing keys on the keyboard. These actions help automate user input such as typing text, pressing Enter, pressing special keys like Tab, Shift, Control, and performing keyboard shortcuts.

Playwright provides keyboard actions using the keyboard class

type()

The type() method is used to type text into the focused element. It simulates real typing by sending characters one by one.

insertText()

The insertText() method is used to insert text directly into the focused element. It does not simulate real key presses and does not trigger keyboard events like keydown or keyup.

down()

The `down()` method is used to press and hold a keyboard key. It is useful for key combinations like Shift, Control, or Alt.

up()

The `up()` method is used to release a key that was pressed using the `down()` method.

EXAMPLE:

```
import { test } from '@playwright/test';

test('Keyboard methods example', async ({ page }) => {

  await page.goto('https://demo.playwright.dev/todomvc');

  // Focus input field
  await page.locator('.new-todo').click();

  // type()
  await page.keyboard.type('Hello');

  // insertText()
  await page.keyboard.insertText(' Playwright');

  // down()
  await page.keyboard.down('Shift');

  // press()
  await page.keyboard.press('A');

  // up()
  await page.keyboard.up('Shift');

  // press Enter
  await page.keyboard.press('Enter');

});
```


Radio buttons and checkboxes are form elements that allow users to select options. A checkbox allows selecting multiple options, while a radio button allows selecting only one option from a group. Playwright provides built-in methods to interact with radio buttons and checkboxes easily.

Playwright provides the methods `check()` and `unchecked()` to select and deselect these elements. These methods automatically wait for the element to be visible, enabled, and ready before performing the action.

The `check()` method is used to select a radio button or checkbox. If the element is already selected, Playwright does nothing.

The `unchecked()` method is used to deselect a checkbox. This method works only for checkboxes, not radio buttons, because radio buttons cannot be unselected directly.

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Radio and Checkbox example', async ({ page }) => {

  await page.goto('https://demo.playwright.dev/input');

  // Select checkbox
  const checkbox = page.locator('input[type="checkbox"]').first();
  await checkbox.check();

  // Select radio button
  const radio = page.locator('input[type="radio"]').first();
  await radio.check();

  // Verify both are selected
  await expect(checkbox).toBeChecked();
  await expect(radio).toBeChecked();

});
```

Dropdown Actions in Playwright (Typescript)

Dropdowns are used to select an option from a list of available values.

Playwright provides the `selectOption()` method to handle dropdown elements. This method is used with `<select>` elements and allows selecting options using value, label, or index.

Playwright automatically waits for the dropdown to be visible and ready before selecting the option.

TYPES OF DROPDOWN:

1-Standard dropdown

2-Multi select dropdown

3-Custom dropdown

4- Autosuggest/Searchable dropdown

Standard Dropdown in Playwright (Typescript)

A standard dropdown is a dropdown created using the HTML `<select>` tag. It contains multiple `<option>` elements inside it. This type of dropdown allows users to select one option from the list. Standard dropdowns are easy to identify because they always use the `<select>` tag in the HTML.

Playwright provides the `selectOption()` method to handle standard dropdowns. This method allows selecting an option using its value, visible text (label), or index. Playwright automatically waits for the dropdown to be visible and ready before selecting the option.

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Standard dropdown example', async ({ page }) => {

  // Open webpage
  await page.goto('https://demo.playwright.dev/input');

  // Select option from standard dropdown
  await page.locator('select').selectOption('blue');

  // Verify selected value
  await expect(page.locator('select')).toHaveValue('blue');

});
```

Multi-Select Dropdown in Playwright (Typescript)

A multi-select dropdown is a standard dropdown created using the `<select>` tag with the `multiple` attribute. This type of dropdown allows users to select more than one option at the same time.

In Playwright, multi-select dropdowns are handled using the same `selectOption()` method, but you pass multiple values in an array.

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Multi-select dropdown example', async ({ page }) => {

  // Open dropdown demo page
  await page.goto('https://demo.playwright.dev/input');

  // Select multiple options
  await page.locator('select').selectOption(['red', 'green']);

  // Verify selected values
  await expect(page.locator('select')).toHaveValues(['red', 'green']);

});
```

Custom Dropdown in Playwright (Typescript)

A custom dropdown is a dropdown that is not created using the HTML `<select>` tag. Instead, it is created using elements like `<div>`, `<ul>`, `<li>`, `<span>`, or `<input>`. These dropdowns are designed using Typescript and CSS, and the options appear dynamically when you click or type.

Because custom dropdowns do not use the `<select>` tag, the `selectOption()` method cannot be used. Instead, you must use `click()` to open the dropdown and then `click()` again to select the required option.

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Custom dropdown example', async ({ page }) => {

  // Open website
  await page.goto('https://demo.playwright.dev/todomvc');

  // Click dropdown (example element acting as dropdown)
  await page.locator('.new-todo').click();

  // Type text to simulate dropdown selection
  await page.locator('.new-todo').fill('Custom Option');

  // Press Enter to select option
  await page.keyboard.press('Enter');

  // Verify selection
  await expect(page.locator('.todo-list li')).toBeVisible();

});
```

Auto Suggest Dropdown in Playwright (Typescript)

An auto suggest dropdown is a type of dropdown where suggestions appear automatically when the user starts typing in an input field. These dropdowns are dynamic and are commonly used in search boxes, such as Google search, where suggestions appear based on the entered text.

Auto suggest dropdowns are not created using the `<select>` tag. Instead, they are created using elements like `<div>`, `<ul>`, and `<li>`. Because of this, the `selectOption()` method cannot be used. You must use `fill()` to type text and `click()` to select the desired suggestion.

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Auto suggest dropdown example', async ({ page }) => {

  // Open Google website
  await page.goto('https://www.google.com');

  // Type text in search box
  await page.locator('textarea[name="q"]').fill('playwright');

  // Click on one of the suggested options
  await page.locator('text=playwright tutorial').first().click();

  // Verify results page is opened
  await expect(page).toHaveURL(/playwright/);

});
```

Window Handles in Playwright (Typescript)

Window handles in Playwright are used to manage multiple browser windows or tabs. When you click a link or button, sometimes a new tab or new window opens. Playwright allows you to switch control between the original window and the new window.

Handling New Tab in Playwright (Typescript)

A new tab opens when you click a link or button that launches another page in a separate browser tab. In Playwright, each tab is represented by a **Page object**. To handle a new tab, Playwright provides the `waitForEvent('popup')` method, which waits until the new tab opens and gives control to it.

This allows you to perform actions and validations in the new tab.

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('New tab example', async ({ page }) => {

  // Open website
  await page.goto('https://example.com');

  // Wait for new tab and click link
  const newTabPromise = page.waitForEvent('popup');
  await page.locator('text=More information').click();

  // Get new tab page object
  const newTab = await newTabPromise;

  // Wait for new tab to load
  await newTab.waitForLoadState();

  // Verify new tab title
  await expect(newTab).toHaveTitle(/Example Domain/);

});
```

Frames in Playwright (Typescript)

A frame is a webpage inside another webpage. Frames are created using the `<iframe>` tag in HTML. They are used to embed another page, such as login forms, payment gateways, or external content, inside the main page.

Playwright cannot directly interact with elements inside a frame using the main page locator. You must first switch to the frame and then perform actions.

Playwright provides the `frameLocator()` method to handle frames easily. This method allows Playwright to locate the frame and interact with elements inside it.

1-frame() in Playwright (Typescript)

The `frame()` method in Playwright is used to access a specific frame from the page using the frame name, URL, or other properties. It returns a **Frame object**, which allows you to interact with elements inside that frame.

This method is useful when a webpage contains multiple frames and you want to work with a particular frame.

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Example of frame()', async ({ page }) => {

  // Open webpage with frame
  await page.goto('https://demo.playwright.dev/frames');

  // Get frame using frame name
  const frame = page.frame({ name: 'frame1' });

  // Click button inside frame
  await frame.locator('button').click();

});
```

2-frameLocator() in Playwright (Typescript)

The `frameLocator()` method is used to locate and interact with elements inside an iframe. It is the recommended and easiest way to handle frames in Playwright. Instead of switching to the frame separately, `frameLocator()` directly allows you to find and interact with elements inside the frame.

This method works using the iframe selector, such as id, name, or any locator.

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('frameLocator example', async ({ page }) => {

  // Open webpage with iframe
  await page.goto('https://demo.playwright.dev/frames');

  // Locate frame and click button inside frame
  await page.frameLocator('#frame1').locator('button').click();

});
```

contentFrame() in Playwright (Typescript)

The `contentFrame()` method is used to get the `Frame` object from an `iframe` locator. Once you get the `Frame` object, you can interact with elements inside that frame.

This method is useful when you first locate the `iframe` element and then want to access its content.

EXAMPLE:

```
import { test } from '@playwright/test';

test('contentFrame example', async ({ page }) => {

  // Open webpage with iframe
  await page.goto('https://demo.playwright.dev/frames');

  // Locate iframe and get Frame object
  const frame = await page.locator('#frame1').contentFrame();

  // Click button inside frame
  await frame.locator('button').click();

});
```

Dialogs in Playwright (Typescript)

Dialogs are popup messages that appear on the screen to interact with the user. These dialogs are created using Typescript methods like `alert()`, `confirm()`, and `prompt()`. Playwright provides the `page.on('dialog')` event to handle these dialogs.

When a dialog appears, Playwright must handle it using `accept()` or `dismiss()`. If you do not handle the dialog, the test will stop because the dialog blocks further execution.

`page.on()` in Playwright (Typescript)

The `page.on()` method is used to listen for events that happen on the page. Events are actions such as dialogs appearing, new tabs opening, file downloads starting, or page crashes. This method allows Playwright to detect these events and perform actions when they occur.

EXAMPLE:

```
import { test } from '@playwright/test';

test('page.on dialog example', async ({ page }) => {

  // Open dialog demo page
  await page.goto('https://demo.playwright.dev/dialogs');

  // Listen for dialog event
  page.on('dialog', async dialog => {
    console.log('Dialog message:', dialog.message());
    await dialog.accept();
  });

  // Click button to open dialog
  await page.locator('text=Click me').click();

});
```

page.once() in Playwright (Typescript)

The page.once() method is used to listen for an event only one time. It works the same as page.on(), but the difference is that page.once() handles the event only once and then automatically stops listening.

This is useful when you expect the event to occur only one time, such as a single dialog, popup, or download.

EXAMPLE:

```
import { test } from '@playwright/test';

test('page.once dialog example', async ({ page }) => {

  // Open dialog demo page
  await page.goto('https://demo.playwright.dev/dialogs');

  // Listen for dialog event only once
  page.once('dialog', async dialog => {
    console.log('Dialog message:', dialog.message());
    await dialog.accept();
  });

  // Click button that opens dialog
  await page.locator('text=Click me').click();

});
```

Notifications in Playwright (Typescript)

Notifications are popup messages shown by the browser to inform users about updates, alerts, or messages. These are also called browser notifications or permission notifications. When a website asks for permission to show notifications, Playwright can handle it by granting or denying permission.

This is done using the permissions property inside browser.newContext().

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Notification example using context', async ({ browser }) => {

  // Create browser context with notification permission
  const context = await browser.newContext({
    permissions: ['notifications']
  });

  // Create new page in that context
  const page = await context.newPage();

  // Open website
  await page.goto('https://example.com');

  // Verify page opened
  await expect(page).toHaveTitle(/Example/);

});
```

File Upload and Download in Playwright (Typescript)

Playwright provides built-in methods to upload and download files easily. File upload is used to send a file from your system to the application, and file download is used to save a file from the application to your system.

In this example, Playwright first uploads a single file using `setInputFiles('filePath')`. Then Playwright uploads multiple files by passing an array of file paths to `setInputFiles()`.

After that, Playwright waits for the download event using `waitForEvent('download')` and saves the downloaded file using `saveAs()`.

This example demonstrates how to upload single files, multiple files, and download files in one test script.

EXAMPLE:

```
import { test, expect } from '@playwright/test';

test('Single upload, multiple upload, and download example', async ({ page }) => {

  // Open website
  await page.goto('https://demo.playwright.dev/input');

  // Upload single file
  await page.locator('input[type="file"]').setInputFiles('test-data/sample1.pdf');

  // Upload multiple files
  await page.locator('input[type="file"]').setInputFiles([
    'test-data/sample1.pdf',
    'test-data/sample2.pdf'
  ]);

  // Verify input is visible after upload
  await expect(page.locator('input[type="file"]')).toBeVisible();

  // Wait for download and click download link
  const downloadPromise = page.waitForEvent('download');
  await page.locator('text=Download').click();

  // Get download object
  const download = await downloadPromise;

  // Save downloaded file
  await download.saveAs('downloads/sample.pdf');

});
```

Data Driven Testing:

Data Driven Testing is a testing approach where the same test script is executed multiple times using different sets of input data. Instead of writing separate test scripts for each scenario, the test data is stored separately, and the same test logic is reused with different values. This makes the test more efficient, reusable, and easier to maintain.

In Playwright, data driven testing is commonly done using arrays, loops, or external files like JSON, CSV, or Excel. The test reads the data and runs the same steps for each data set. For example, a login test can be executed with multiple usernames and passwords using one test script.

This approach helps increase test coverage because multiple scenarios such as valid input, invalid input, and edge cases can be tested easily. It also reduces code duplication and makes the automation framework more organized and scalable.

Data Driven Testing using Excel File

In Data Driven Testing using an Excel file, the test data is stored in an Excel sheet instead of writing it directly in the test script. The Playwright test reads the data from the Excel file and executes the same test multiple times using different values from the Excel sheet.

The Excel file usually contains rows and columns where each row represents one test data set, and each column represents a field such as username, password, email, or any other input. Playwright reads this data and uses it in the test execution.

To read Excel files in Playwright with Typescript, a library such as `xlsx` is used. This library helps read data from Excel files and convert it into a format like JSON, which Playwright can easily use.

This approach is very useful when there is a large amount of test data. Instead of modifying the test script every time, you can simply update the Excel file. It also makes the test easier to maintain, more organized, and suitable for real-world automation frameworks.

Using Excel for data driven testing improves reusability, reduces code duplication, and allows testers to manage test data separately from the test logic.

Steps for Data Driven Testing using Excel File (exceljs) in Playwright

1. Install the **exceljs** library in your Playwright project. Open the terminal and run:
`npm install exceljs`
This library allows Playwright to read data from Excel files.
2. Install the **Excel Viewer extension** in VS Code. Go to Extensions, search for **Excel Viewer**, and install it. This helps you open and view Excel files inside VS Code.
3. Create an Excel file in your project folder, for example testdata.xlsx. Add column names like Username and Password, and enter test data in rows.
4. Import the exceljs library in your Playwright test script. This allows your script to access and read the Excel file.
5. Create a workbook object using exceljs. The workbook represents the Excel file.
6. Load the Excel file into the workbook. This step connects your script to the Excel file.
7. Access the worksheet from the workbook. The worksheet contains the test data stored in rows and columns.
8. Read the data from each row of the worksheet. Extract values like username and password from the cells.
9. Store the extracted data in variables or arrays so that Playwright can use it during test execution.
10. Use the data in your Playwright test script to perform actions such as filling forms, logging in, or validating results.
11. Run the test so Playwright executes the same test multiple times using different data from the Excel file.

EXAMPLE:

```
import { test, expect } from '@playwright/test';
import ExcelJS from 'exceljs';

test('Data Driven Testing using Excel', async ({ page }) => {

    // Step 1: Create workbook object
    const workbook = new ExcelJS.Workbook();

    // Step 2: Load Excel file
    await workbook.xlsx.readFile('testdata.xlsx');

    // Step 3: Get worksheet
    const sheet = workbook.getWorksheet('Sheet1');

    // Step 4: Read data from rows
    for (let i = 2; i <= sheet.rowCount; i++) {

        const username = sheet.getRow(i).getCell(1).value;
        const password = sheet.getRow(i).getCell(2).value;

        // Step 5: Open website
        await page.goto('https://practicetestautomation.com/practice-test-login/');

        // Step 6: Use Excel data
        await page.locator('#username').fill(username);
        await page.locator('#password').fill(password);

        // Step 7: Click login
        await page.locator('#submit').click();

        // Step 8: Verification (example)
        await expect(page).toHaveURL(/practice-test-login/);

    }

});
```


Steps to Write Data into Excel File using exceljs

1. Install the **exceljs** library using the command `npm install exceljs`.
2. Import the **exceljs** library into your Typescript file.
3. Create a new workbook using `new ExcelJS.Workbook()`. This represents the Excel file.
4. Add a worksheet using `workbook.addWorksheet('Sheet1')`. This creates a sheet inside the Excel file.
5. Write data into the worksheet using `sheet.addRow()`. Each `addRow()` represents one row in Excel.
6. Save the Excel file using `workbook.xlsx.writeFile('filename.xlsx')`. This creates the Excel file in your project folder.
7. After saving, open the Excel file to verify that the data is written successfully.

JSON File in Playwright (Typescript)

A JSON file is used to store test data in a structured format using key and value pairs. JSON stands for Javascript Object Notation. It is commonly used in data driven testing because it is simple, readable, and easy to use in Playwright.

In Playwright, JSON files are used to store data such as username, password, URLs, and other test inputs. Playwright reads the data from the JSON file and uses it in the test script.

Steps to Create a JSON File for Playwright

1. Open your Playwright project folder in VS Code.
2. Right click on the folder where you want to create the file (for example, inside the tests folder or project root).
3. Click on **New File**.
4. Give the file name with .json extension, for example: `testdata.json`
5. Open the file and write data in **JSON format** using key and value pairs.
6. Save the file using **Ctrl + S**.

7. Now this JSON file is ready, and you can import and use it in your Playwright test script.

Steps to Use JSON File in Playwright

1. Create a JSON file in your project folder, for example testdata.json.
2. Write test data in the JSON file using key and value format.
3. Save the JSON file.
4. Open your Playwright test script.
5. Import the JSON file using the import statement.
Example: `import data from './testdata.json';`
6. Access the JSON data using the key name.
Example: `data.username, data.password.`
7. Use the JSON data in your test, such as filling input fields or performing actions.
8. Run the Playwright test. The test will use data from the JSON file during execution.

Page Object Model (POM) in Playwright (Typescript)

Page Object Model (POM) is a design pattern used in automation testing to organize and manage code. In this approach, each webpage is represented as a separate class file, and all the locators and methods related to that page are stored inside that class.

Instead of writing locators and actions directly in the test script, they are written inside the page class. The test script only calls the methods from the page class. This makes the code clean, reusable, and easy to maintain.

For example, a login page will have a LoginPage class that contains locators for username, password, and login button, and methods like `enterUsername()`, `enterPassword()`, and `clickLogin()`.

The test script imports the LoginPage class and uses its methods to perform the test.

The main advantage of POM is that if there is any change in the webpage, you only need to update the page class file, not all test scripts. It also improves code readability, reusability, and maintainability.

POM is commonly used in real-world Playwright frameworks to create structured and scalable automation projects.

Steps to Implement Page Object Model (POM) in Playwright

1. Create a separate folder in your project, usually named **pages**. This folder will store all page class files.
2. Create a class file for each webpage inside the pages folder. For example, create LoginPage.js for the login page.
3. Inside the class file, create a class and define the constructor. Pass the page object in the constructor so it can be used inside the class.
4. Define all locators inside the class. These locators represent elements like username field, password field, and login button.
5. Create methods inside the class to perform actions such as entering username, entering password, and clicking the login button.
6. Export the class so it can be used in the test script.
7. Go to your test script file and import the page class.
8. Create an object of the page class by passing the Playwright page object.
9. Call the methods from the page class inside your test script to perform actions.
10. Run the test. The test script will use the page class methods instead of directly using locators.

These steps help organize the automation code, improve reusability, and make the framework easier to maintain.

EXAMPLE:

```
class LoginPage {  
  
  constructor(page) {  
    this.page = page;  
    this.username = '#username';  
    this.password = '#password';  
    this.loginButton = '#submit';  
  }  
  
  async navigate() {  
    await this.page.goto('https://practicetestautomation.com/practice-test-login/');  
  }  
  
  async login(user, pass) {  
    await this.page.locator(this.username).fill(user);  
    await this.page.locator(this.password).fill(pass);  
    await this.page.locator(this.loginButton).click();  
  }  
  
}  
export default LoginPage
```

Advantages of Page Object Model (POM)

1. Improves code readability

The test script becomes clean and easy to understand because locators and actions are stored in separate page class files.

2. Increases code reusability

The same page class methods can be used in multiple test scripts, so you do not need to write the same code again.

3. Easy maintenance

If there is any change in the webpage, you only need to update the locator in the page class file, not in all test scripts.

4. Reduces code duplication

Locators and methods are written only once in the page class, which avoids repeating the same code in multiple tests.

5. Better organization of code

The project structure becomes organized because each page has its own class file.

6. Improves scalability

POM makes it easy to add more pages and tests when the project grows.

7. Separates test logic and page logic

The test script contains only test steps, and the page class contains locators and actions. This makes the framework more structured.

8. Makes framework suitable for real-world projects

Most automation frameworks use POM because it helps manage large and complex applications efficiently.